University of Central Punjab

Faculty of Information Technology & Computer Science

Department of Applied Computing & Technologies

Subject: Agent Based Modeling

Assignment 2

Personal Expense Management Agent

Scenario-Based Intelligent Agent Analysis: From Environment and Agent Architecture to Reasoning, Planning and Replanning

Student Name	Roll Number
Ahmed Ali	L1F23BSAI0092
Noman Tariq Butt	L1F23BSAI0095
Mohsin Sajjad	L1F23BSAI0117

Submitted to: Sir Zeeshan Hussain

Due Date: 06/11/2026

1. Introduction

This report presents a Personal Expense Management Agent, a Python-based AI agent that helps a user categorize transactions, remember budgets and goals, analyze spending patterns and understand basic finance concepts. It addresses CLO-2 by combining an LLM, prompt engineering, short-term and long-term memory, Retrieval-Augmented Generation (RAG) and a Model Context Protocol (MCP) tool server in one agent. All data is synthetic, and consequential actions require explicit human confirmation.

2. Problem and Environment

The agent operates in a personal-finance environment. Its percepts are user messages and its tools; its actions are tool calls and replies.

Element	Description
Environment	Personal finance: a transaction history (synthetic CSV), saved budgets and goals, and a finance knowledge base.
Percepts	User questions, tool outputs, retrieved document chunks, stored memory.
Actions	Query spending via MCP tools, search the knowledge base, save or recall memory, answer, ask for confirmation.
Goal	Give accurate, grounded and safe budgeting help without inventing figures.

3. System Architecture

One primary agent, driven by an LLM and a system prompt, selects between three paths: the RAG tool for concepts, the MCP client for spending data, and the memory tools for user preferences. Short-term memory keeps the current conversation; long-term memory persists across sessions.

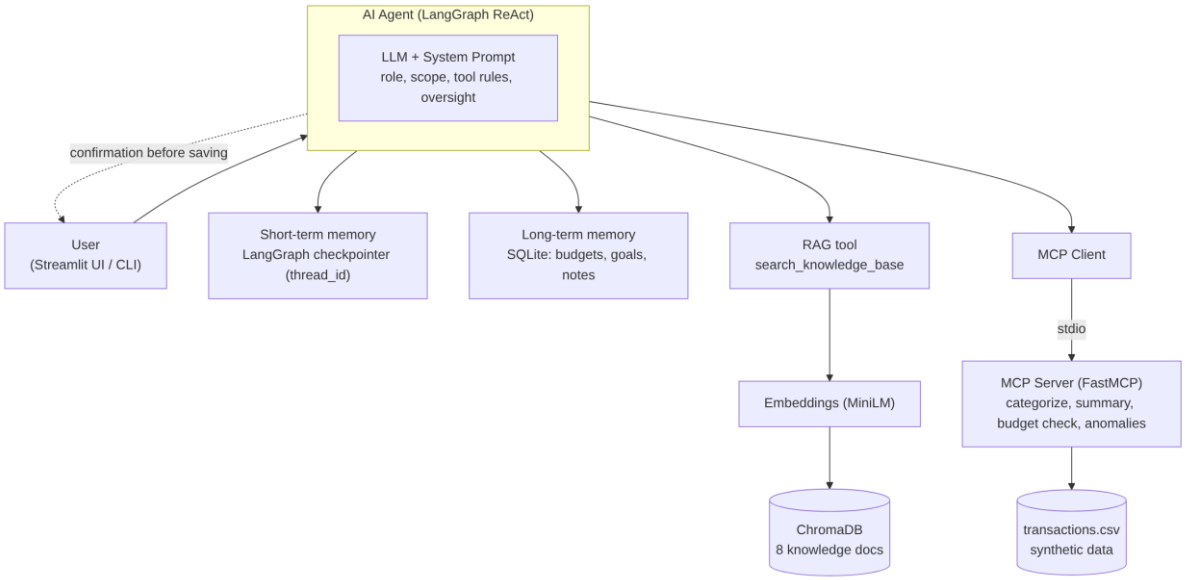


Figure 1: Architecture of the Personal Expense Management Agent

3.1 Components

Component	Implementation
AI Agent + LLM	Python ReAct-style agent (LangGraph) using a configurable chat model with tool calling.
Short-term memory	Session-based conversation history stored by a checkpointer and keyed by a session (thread) ID, so follow-up questions refer to earlier turns.
Long-term memory	SQLite database holding budgets, goals and notes per user; recalled in a new session.
RAG	8 knowledge documents are chunked, embedded and stored in ChromaDB; the agent retrieves chunks and cites source files.
MCP	FastMCP server exposing expense tools, connected through an MCP client over stdio.
Prompt engineering	System prompt defining role, scope, tool-selection rules, evidence use, memory handling and uncertainty behavior.
User interface	Streamlit chat app with session control, an evidence panel and a memory confirmation panel.

4. Implementation Details

4.1 RAG pipeline

The knowledge base has eight documents: budgeting methods, expense category taxonomy, emergency fund, debt management, overspending and anomaly detection, subscription audit, cost-saving tips, and agent policy and safety. Documents are split into chunks (about 500 characters with overlap), embedded with a sentence-transformer model and stored in ChromaDB with source metadata. At query time, the top matching chunks are returned and the answer cites the source file.

4.2 MCP server

The MCP server exposes deterministic tools that read the synthetic transaction file:

- **categorize_transaction:** classifies a merchant into a category with a confidence level.
- **monthly_summary:** totals spending per category for a month.
- **check_budget:** compares spending to a budget (on track, warning, overspent).
- **detect_anomalies:** finds unusually large transactions and possible duplicate charges.

The agent connects as an MCP client, so each invocation and its result is visible in the evidence panel. A plain HTTP API is not used.

4.3 Memory

Short-term memory is the per-session message history. Long-term memory is a SQLite store of the user's budgets and goals, retrieved when a new session starts. RAG documents and long-term memory serve different purposes: documents hold shared general knowledge, while memory holds user-specific facts.

4.4 Prompt engineering

Prompt section	Purpose
Role and scope	Keeps the agent focused on expense help; no investment, tax or legal advice.
Tool-selection rules	User spending figures come from MCP tools; concepts come from RAG; preferences from memory.
Evidence use	Cite source documents and tool outputs; never invent amounts.
Memory handling	Recall memory at session start; save only after user confirmation.
Failure behavior	State when documents do not cover a question; report tool errors plainly.

4.5 Planning and replanning

For goals such as reducing monthly spending, the agent plans in steps (summarize spending, retrieve saving tips, propose cuts). If the user reports a change, for example a rent increase, the agent updates its assumptions, revises the plan and asks for confirmation before changing stored memory.

5. Results and Output Screenshots

5.1 RAG: grounded answer

The user asked about the 50/30/20 rule. The agent retrieved the budgeting document and answered with the needs, wants and savings split, citing 01_budgeting_methods.md. The evidence panel shows the tool call used.

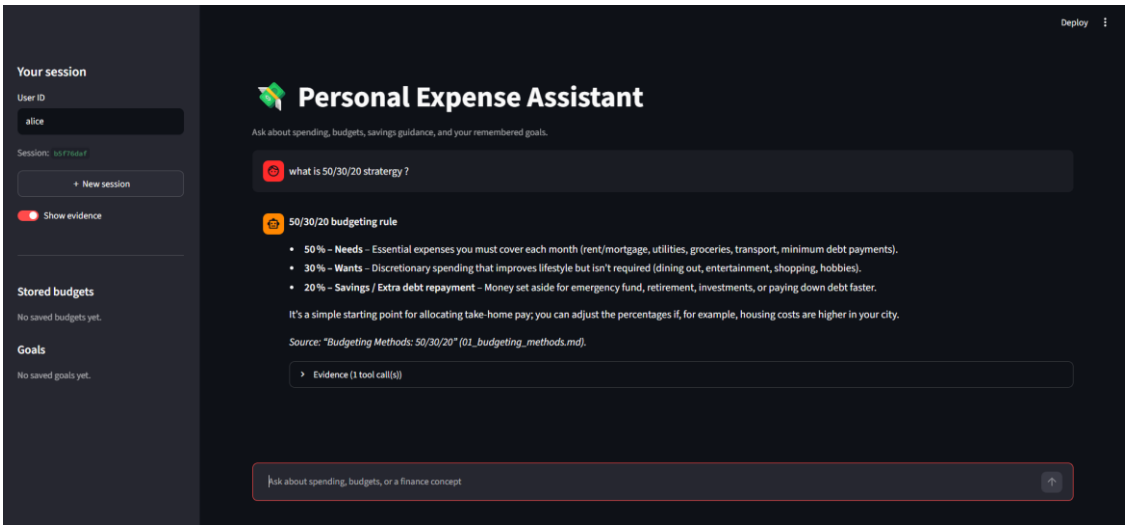


Figure 2: RAG response with source citation

5.2 Long-term memory with human confirmation

When the user asked to store a budget, the agent first requested the missing details (amount, currency, name), then asked for confirmation. The memory change appears in the sidebar with Yes/No buttons and nothing is saved until the user confirms. The sidebar also shows goals already stored for this user.

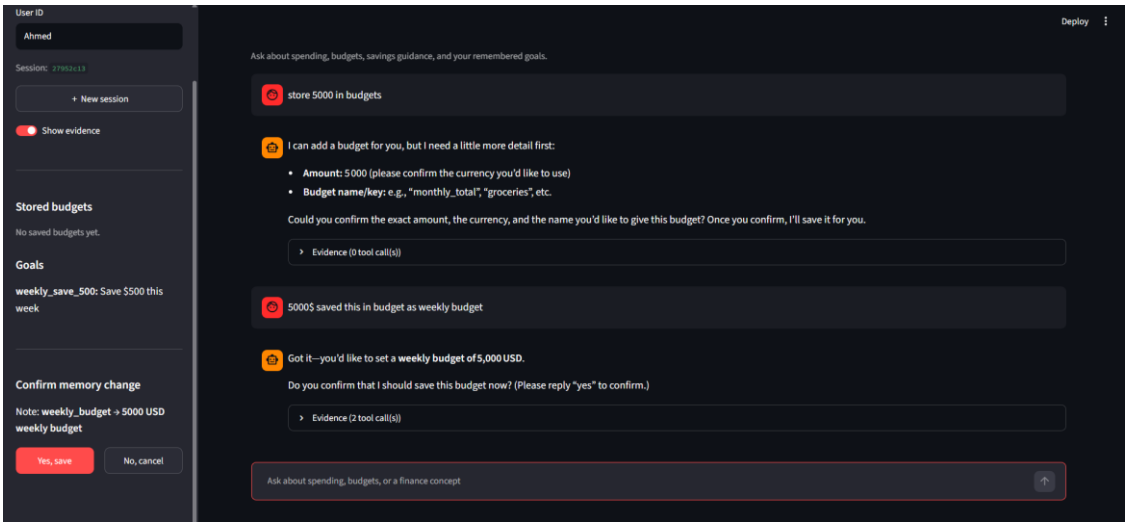


Figure 3: Confirmation required before saving a budget to long-term memory

6. Test Summary

Requirement	How it was demonstrated
Short-term memory	Multi-turn conversation with follow-up questions referring to earlier answers.
Long-term memory	Budget saved in one session and recalled in a new session from SQLite.
RAG	Questions answered from the 8 documents with source citation (Figure 2).
MCP	Real tool invocation (for example anomaly detection) with logged arguments and results.
Human oversight	Budget changes require explicit confirmation (Figure 3).
Error handling	Invalid inputs (such as a wrong month) return clear messages instead of crashing.

7. Limitations and Ethics

- Data is synthetic; the agent is an educational tool, not financial advice.
- The LLM may occasionally choose a different tool or ask clarifying questions; temperature is kept low to improve repeatability.
- Passwords, card numbers and OTPs are never requested or stored.
- Currency handling relies on what the user states (for example, the stored budget in Figure 3 is in USD while the data is in PKR); a fixed default currency would improve consistency.

8. Conclusion

The project delivers a single Python agent that integrates an LLM, engineered prompts, short- and long-term memory, a RAG pipeline and a working MCP server. It grounds its answers in documents and tool outputs and asks for human confirmation before consequential actions, meeting the assignment requirements.